

同济大学计算机系
计算机组成原理课程设计实验报告



题目：31 条 MIPS 指令 CPU 设计

学 号 2252431
姓 名 孙骁远
专 业 计算机科学与技术
授课老师 秦国峰

日期：2024.06.10

一、实验环境部署与硬件配置说明

使用 verilog 建模，实现 31 条 MIPS 指令的 CPU 设计仿真和验证仿真结果

开发平台：vivado 2016.2

二、实验的总体结构

1、31 条 MIPS 指令 CPU 的总体结构

CPU 主要由数据通路和控制部件两部分组成。数据通路是指是指在指令执行过程中，数据所经过的路径和路径上所涉及的功能部件；而控制部件根据每条指令的不同功能，控制取指令、分析指令、执行指令等操作保证正确的指令执行流程。

CPU 功能主要包括以下几点

- (1) 指令控制：完成取指令、分析指令和执行指令的操作。
- (2) 操作控制：它管理每条指令的操作信号，把信号送到相应的部件，从而控制这些部件按指令完成动作。
- (3) 时间控制：为每条指令按时间顺序提供应有的控制信号
- (4) 数据加工：对数据进行算数、逻辑计算
- (5) 中断处理：对特殊请求进行处理，防止异常情况造成的损失

本次实验所有的指令都在一个时钟周期内完成，故称为单周期 CPU。在每条指令执行过程中数据通路的任何资源都不能并行利用，因此，任何需要多次使用的资源都需要设置多个。执行过程大致可分为取指周期、指令译码、间址周期、执行周期四个阶段。

2、指令格式

指令是一组有意义的二进制代码，负责控制机器语言，分为操作码和地址码两部分。分为 3 种指令类型。

(1) .R 型指令

Mnemonic Symbol	Format						Sample
Bit #	31..26	25..21	20..16	15..11	10..6	5..0	
R-type	op	rs	rt	rd	shamt	func	
add	000000	rs	rt	rd	0	100000	add \$1,\$2,\$3
addu	000000	rs	rt	rd	0	100001	addu \$1,\$2,\$3
sub	000000	rs	rt	rd	0	100010	sub \$1,\$2,\$3
subu	000000	rs	rt	rd	0	100011	subu \$1,\$2,\$3
and	000000	rs	rt	rd	0	100100	and \$1,\$2,\$3
or	000000	rs	rt	rd	0	100101	or \$1,\$2,\$3
xor	000000	rs	rt	rd	0	100110	xor \$1,\$2,\$3
nor	000000	rs	rt	rd	0	100111	nor \$1,\$2,\$3
sll	000000	rs	rt	rd	0	101010	sll \$1,\$2,\$3
sllw	000000	rs	rt	rd	0	101011	sllw \$1,\$2,\$3
sllr	000000	0	rt	rd	shamt	000000	sllr \$1,\$2,10
srl	000000	0	rt	rd	shamt	000010	srl \$1,\$2,10
srr	000000	0	rt	rd	shamt	000011	srr \$1,\$2,10
slv	000000	rs	rt	rd	0	000100	slv \$1,\$2,\$3
srlv	000000	rs	rt	rd	0	000110	srlv \$1,\$2,\$3
srrv	000000	rs	rt	rd	0	000111	srrv \$1,\$2,\$3
jr	000000	rs	0	0	0	001000	jr \$31

(2) .I 型指令

Bit #	31..26	25..21	20..16	15..0	
I-type	op	rs	rt	immediate	
addi	001000	rs	rt	Immediate(- ~ +)	addi \$1,\$2,100
addiu	001001	rs	rt	Immediate(- ~ +)	addiu \$1,\$2,100
andi	001100	rs	rt	Immediate(0 ~ +)	andi \$1,\$2,10
ori	001101	rs	rt	Immediate(0 ~ +)	andi \$1,\$2,10
xori	001110	rs	rt	Immediate(0 ~ +)	andi \$1,\$2,10
lw	100011	rs	rt	Immediate(- ~ +)	lw \$1,10(\$2)
sw	101011	rs	rt	Immediate(- ~ +)	sw \$1,10(\$2)
beq	000100	rs	rt	Immediate(- ~ +)	beq \$1,\$2,10
bne	000101	rs	rt	Immediate(- ~ +)	bne \$1,\$2,10
sli	001010	rs	rt	Immediate(- ~ +)	sli \$1,\$2,10
sliu	001011	rs	rt	Immediate(- ~ +)	sliu \$1,\$2,10
lui	001111	00000	rt	Immediate(- ~ +)	Lui \$1, 10

(3) .J 型指令

Bit #	31..26	25..0	
J-type	op	Index	
j	000010	address	j 10000
jal	000011	address	jal 10000

R 型指令主要负责寄存器之间的逻辑和算术运算，I 型指令执行与立即数相关的操作，J 型指令只有两条，主要用于实现无条件跳转，从而改变程序的执行流程。

其中：

op: 31 位~26 位，6 位操作码，决定执行何种指令，R 型指令的操作码均为 000000，其余指令操作码各不相同。

rs: 5 位地址码，代表第一个源操作数地址，只读。

rt: 5 位地址码，代表第二个源操作数地址，可读可写。

rd: 5 位地址码，代表目的操作数地址，只写。

shamt: 5 位数据，专用于移位指令中代表位移量。

func: 6 位功能码，与操作码配合使用，在 R 类指令中，操作码均为 0，用不同的功能码区分执行何种指令。

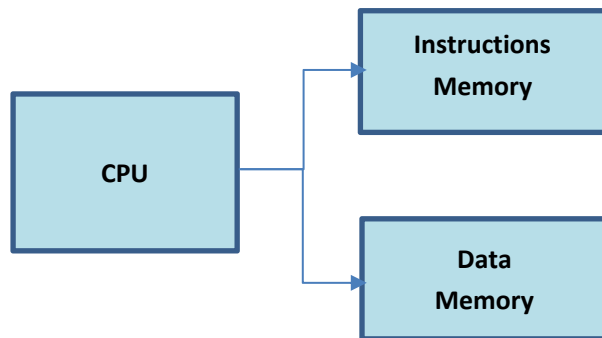
immediate: 16 位立即数，用作无符号的逻辑操作数、有符号的算数操作数、数据加载，数据保存指令的数据地址字节偏移量和分支指令中相对程序计数器的有符号偏移量。

index: 26 位地址码，用于 J 类指令中作 pc 跳转的地址。

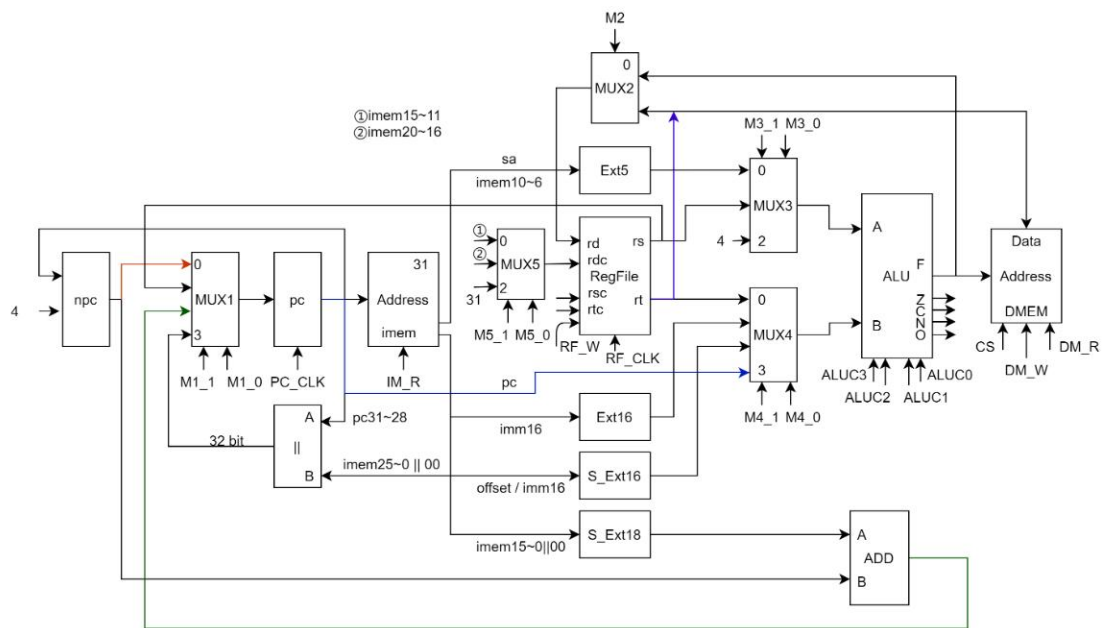
三、 总体架构部件的解释说明

1、CPU 总体结构部件的解释说明

整体框架示意图如下：

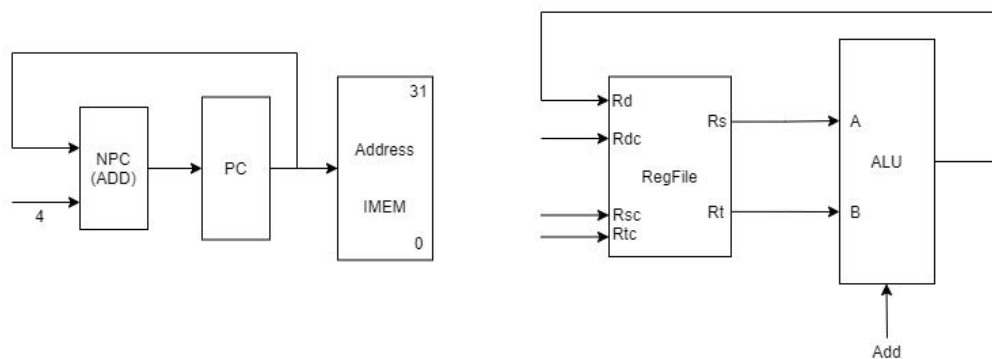


整体接线图如下：

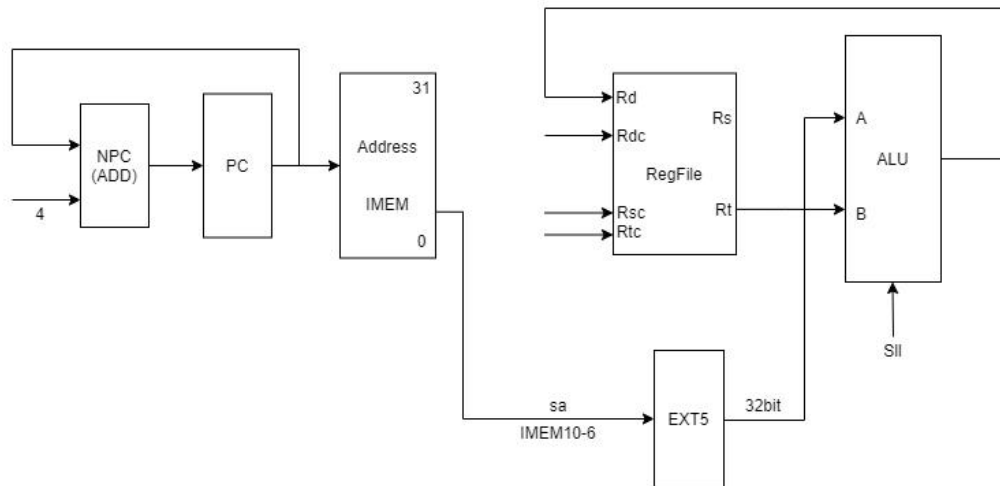


其中每条指令的通路图如下所示：

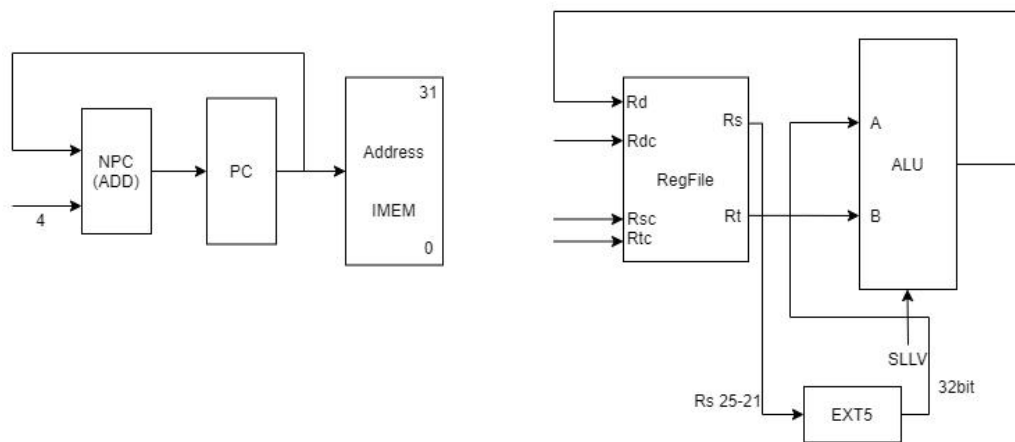
(1) ADD/ADDU/SUB/SUBU/AND/OR/XOR/NOR/SLT/SLTU



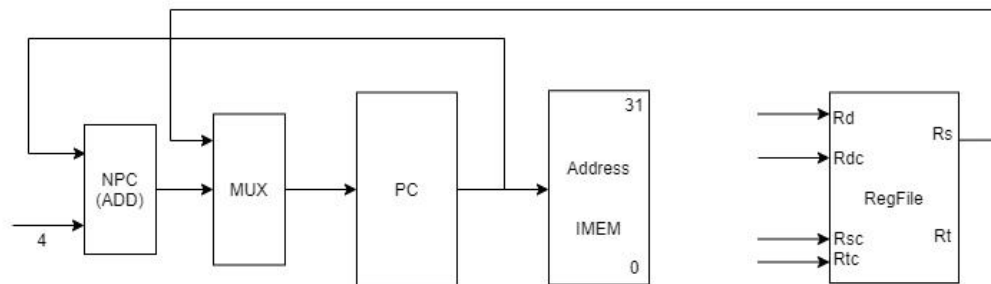
(2) SLL/SRL/SRA



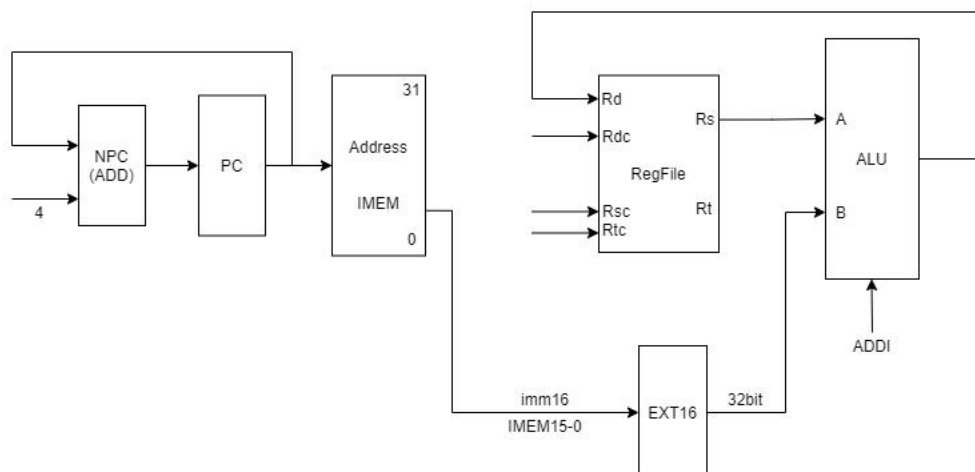
(3) SLLV/SRLV/SRAV



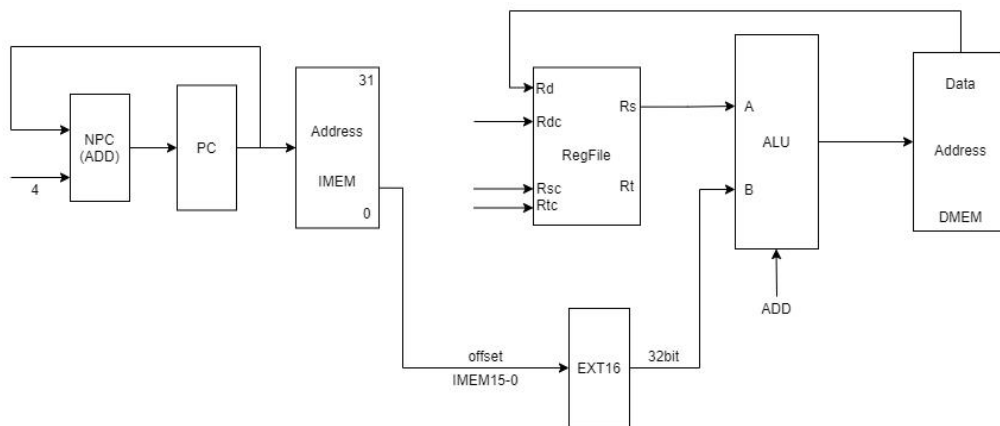
(4) JR



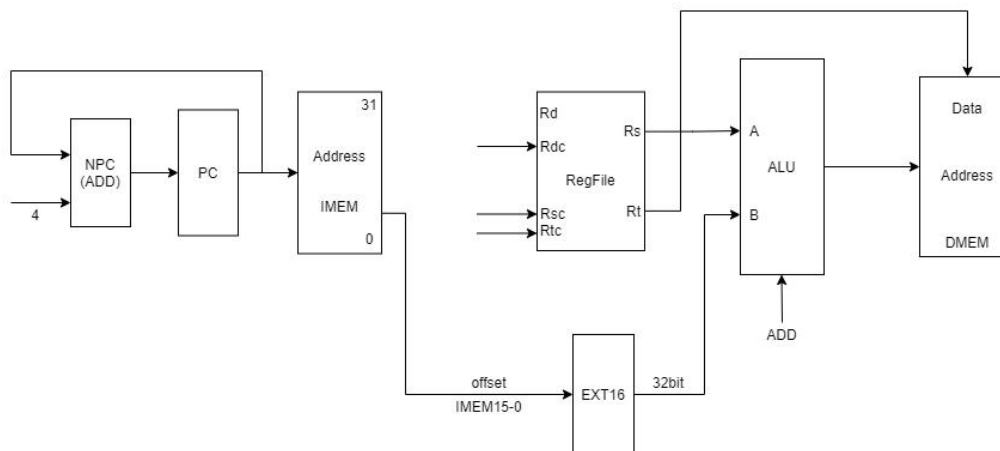
(5) ADDI/ADDIU/ANDI/ORI/XORI



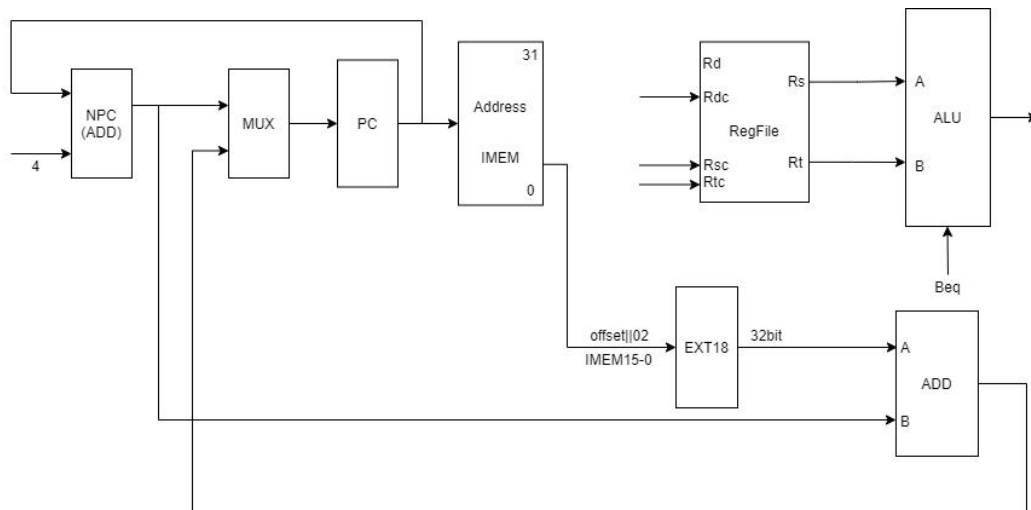
(6) LW



(7) SW



(8) BEQ/BNE



(9) SLTI/SLTIU/LUI

四、实验仿真过程

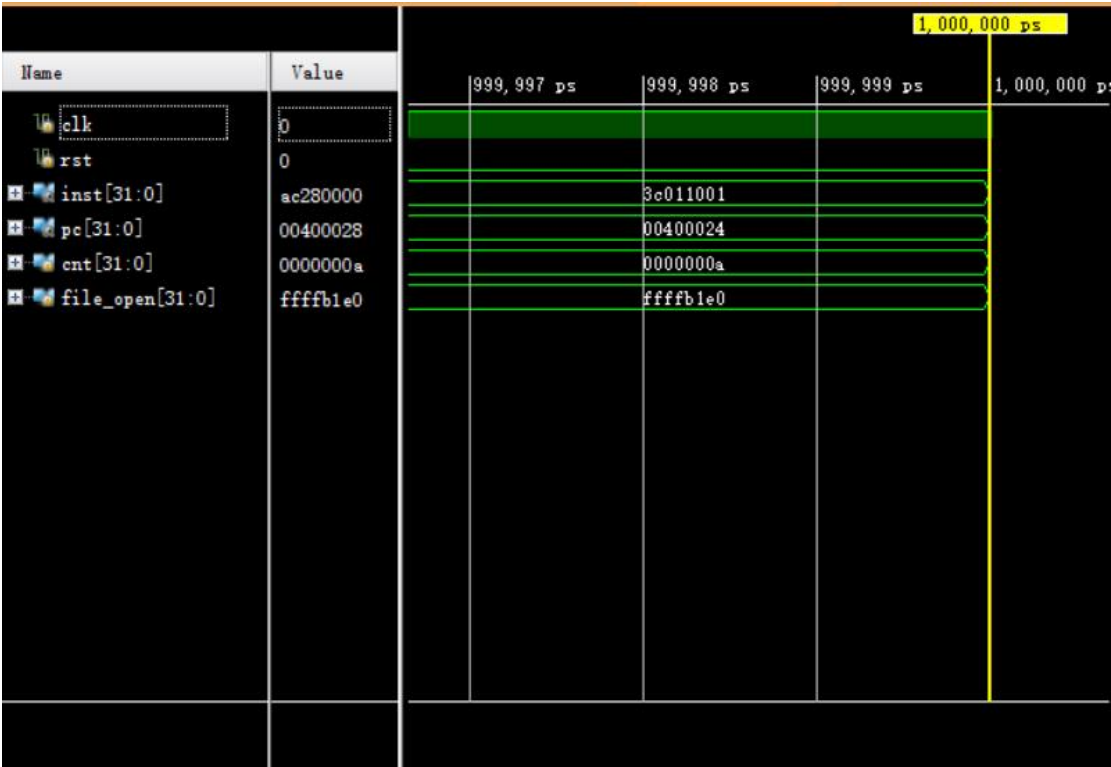
1、CPU 的仿真过程

在 testbench 文件中使用

- (1) file_open = \$fopen("D:\\output.txt", "a+");
- (2) \$fdisplay 函数
- (3) \$fclose(file_open);

这样的流程，在运行仿真过程的同时，生成好了一份存储寄存器数值的测试结果文件。

2、vivado 仿真界面



五、实验仿真的波形图及某时刻寄存器值的物理意义

1、仿真波形图及某时刻寄存器值的物理意义

根据以上仿真界面，其中变量的含义如下：

clk:时钟信号，频率为 50MHz

rst:复位信号，初始化系统，确保从已知状态开始仿真

inst: 要执行的指令码，通过\$fdisplay 函数将指令地址和寄存器值写入文件

pc:下一条指令地址

cnt:计数器，记录已经执行几条指令

file_open:打开文件的句柄

仿真文件将通用寄存器的值写入文件，每个寄存器的值表示某一时刻的数据状态，例如

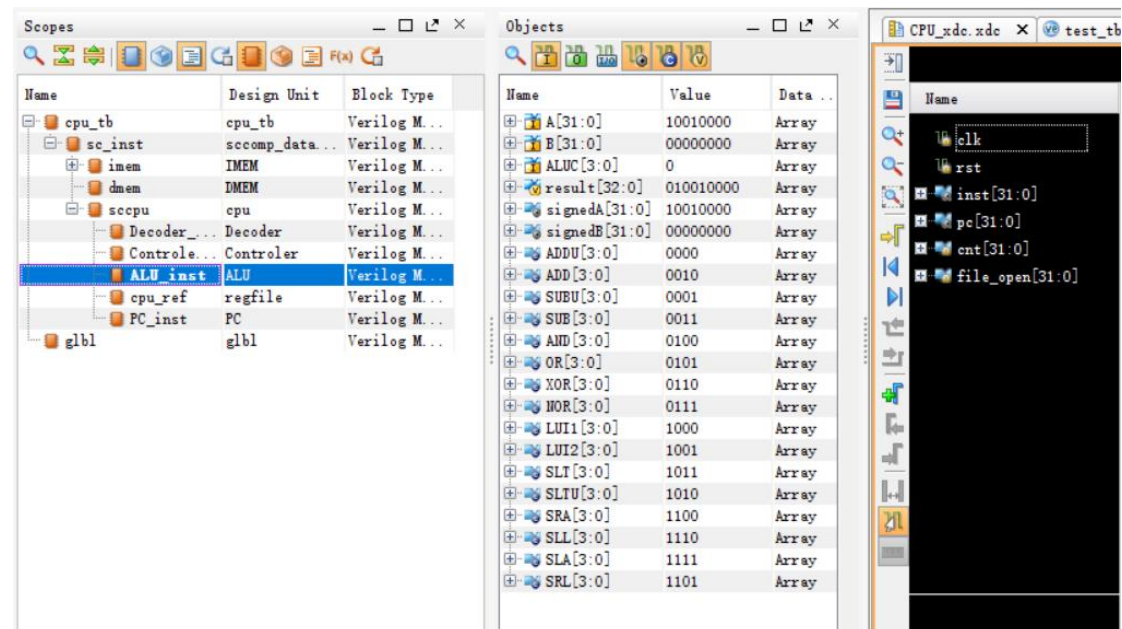
\$zero: 始终为 0 的寄存器

\$at: 保留给汇编器临时使用

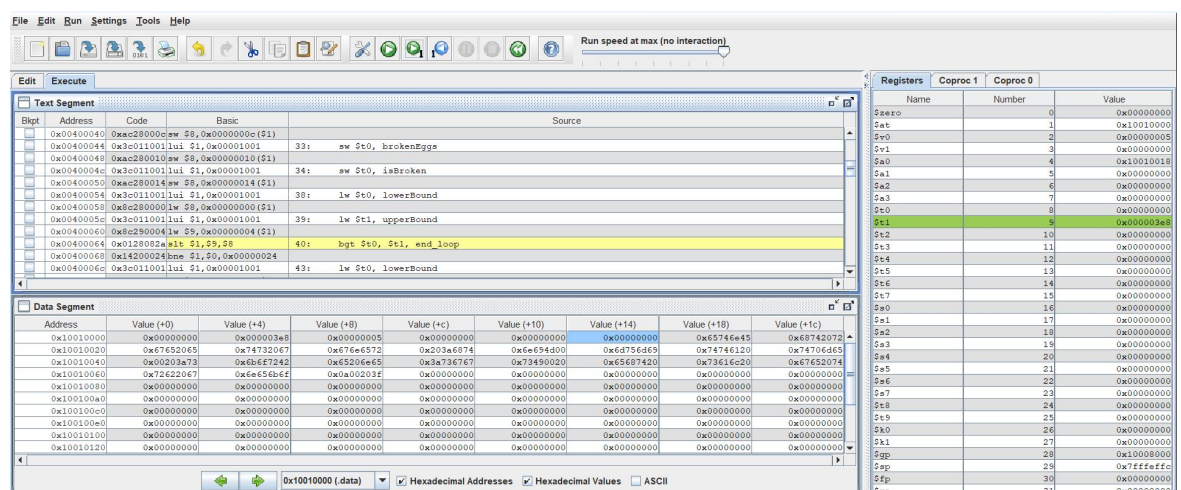
\$v0, \$v1: 用于存储函数返回值

这些寄存器的值在仿真过程中会随着指令的执行而变化，反映了程序的执行状态。

例如：下图是 ALU 模块中每个控制信号和运算结果的值



比萨塔摔鸡蛋游戏模型。两个同学在可变换层数的比萨塔上摔鸡蛋，一个同学秘密设定同一批鸡蛋耐摔值；另一个同学在指定层高的比萨塔拿着鸡蛋往下摔，用最少的摔次数和摔破的鸡蛋数求出鸡蛋的耐摔值。假定在耐摔值的楼层及其下面楼层，鸡蛋摔不破，可以重复使用，否则鸡蛋摔破。要求模型的算法输出包括：摔的总次数、摔的总鸡蛋数、最后摔的鸡蛋是否摔破。请使用 MIPS 指令汇编设计该验证模型的算法，并利用编译器生成 MIPS 指令集可执行目标程序。

[illegible]

将模拟器中 31 条指令与 testbench 后生成的文件对比（限于篇幅，并未把所有寄存器数据都列出）

Text Segment				
Bkpt	Address	Code	Basic	Source
	0x00400060	0x8c290004	lw \$9,0x00000004(\$1)	
	0x00400064	0x0128082a	slt \$1,\$9,\$8	40: bgt \$t0, \$t1, end_loop
	0x00400068	0x14200024	bne \$1,\$0,0x00000024	
	0x0040006c	0x3c011001	lui \$1,0x00001001	43: lw \$t0, lowerBound

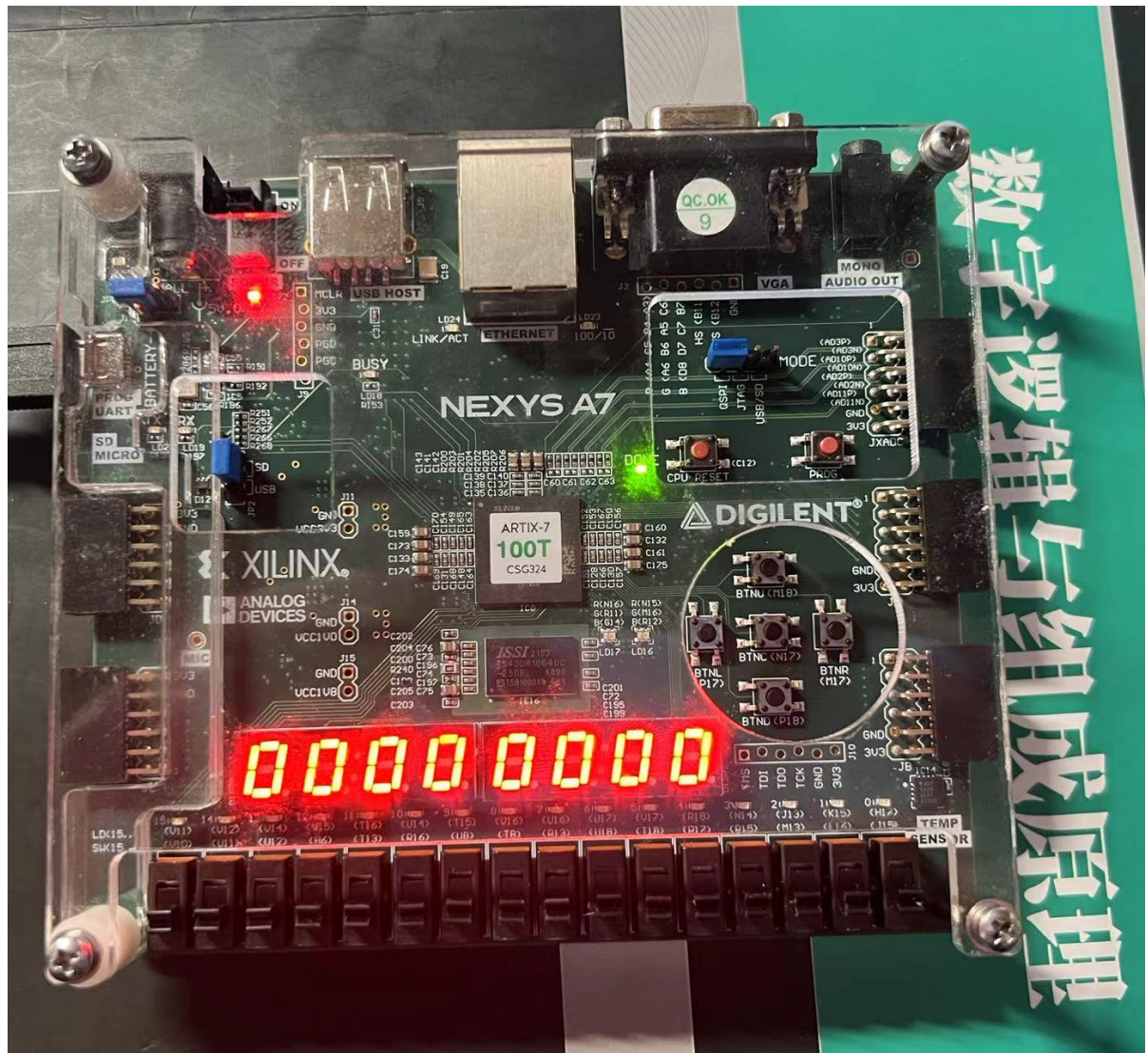
Instr_addr = 0128082a
\$zero = 00000000
\$at = 10010000
\$v0 = 00000005
\$v1 = 00000000
\$a0 = 10010018
\$a1 = 00000000
\$a2 = 00000000
\$a3 = 00000000
\$t0 = 00000000
\$t1 = 000003e8
\$t2 = 00000000
\$t3 = 00000000
\$t4 = 00000000
\$t5 = 00000000
\$t6 = 00000000
\$t7 = 00000000
\$s0 = 00000000
\$s1 = 00000000
\$s2 = 00000000
\$s3 = 00000000
\$s4 = 00000000
\$s5 = 00000000
\$s6 = 00000000
\$s7 = 00000000
\$t8 = 00000000
\$t9 = 00000000
\$k0 = 00000000
\$k1 = 00000000
\$gp = 10008000
\$sp = 7ffffc
\$fp = 00000000
\$ra = 00000000
\$pc = 00400064

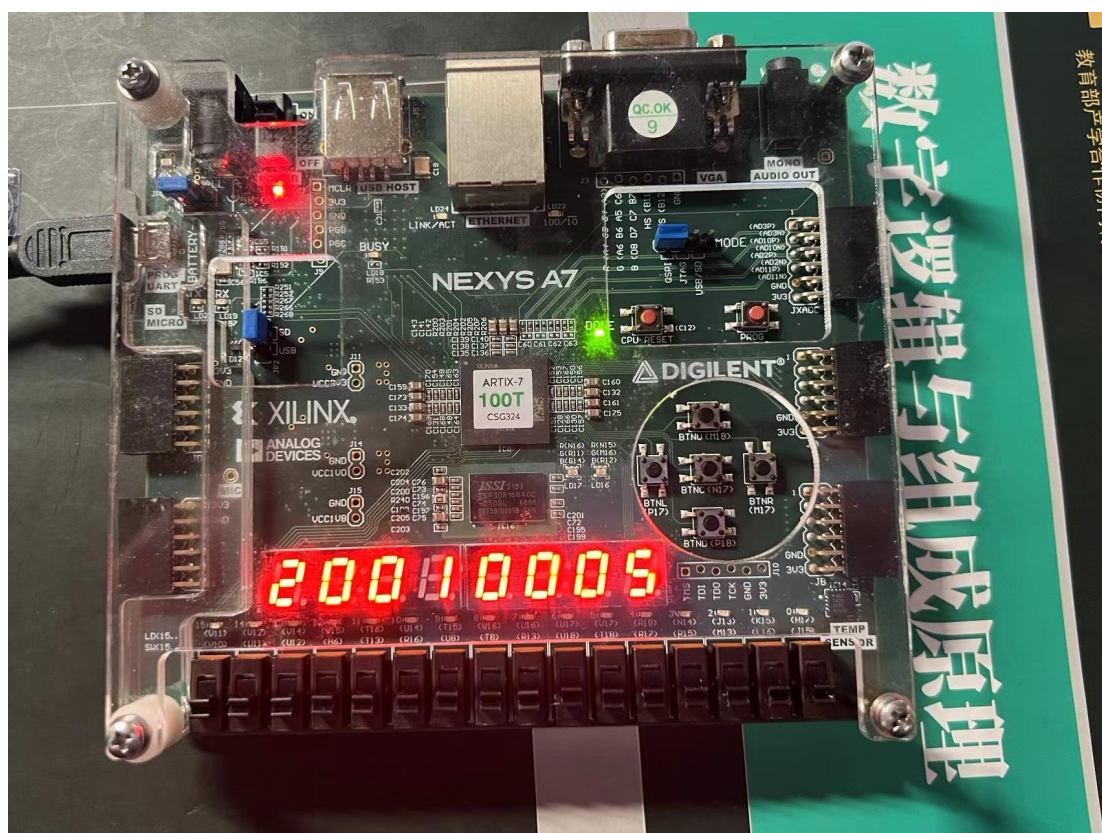
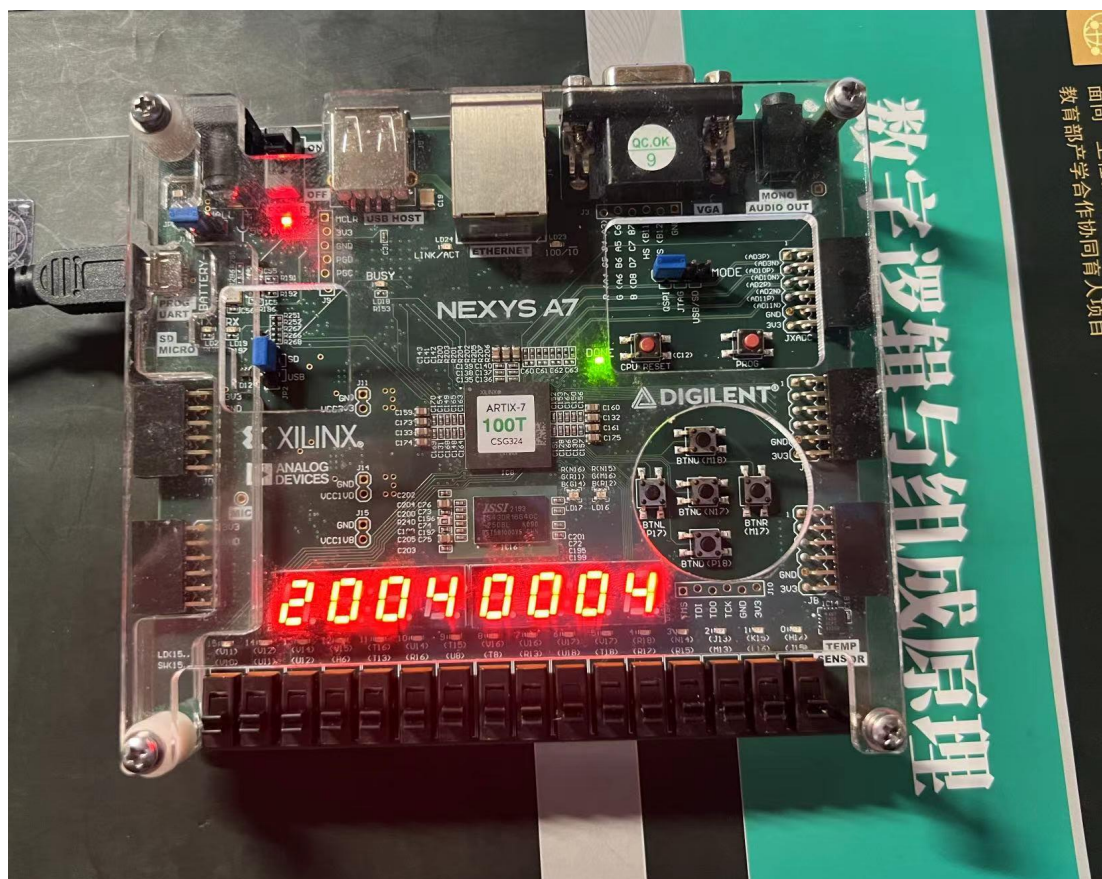
Registers			Coproc 1	Coproc 0
Name	Number	Value		
\$zero	0	0x00000000		
\$at	1	0x10010000		
\$v0	2	0x00000005		
\$v1	3	0x00000000		
\$a0	4	0x10010018		
\$a1	5	0x00000000		
\$a2	6	0x00000000		
\$a3	7	0x00000000		
\$t0	8	0x00000000		
\$t1	9	0x000003e8		
\$t2	10	0x00000000		
\$t3	11	0x00000000		
\$t4	12	0x00000000		
\$t5	13	0x00000000		
\$t6	14	0x00000000		
\$t7	15	0x00000000		
\$s0	16	0x00000000		
\$s1	17	0x00000000		
\$s2	18	0x00000000		
\$s3	19	0x00000000		
\$s4	20	0x00000000		
\$s5	21	0x00000000		
\$s6	22	0x00000000		
\$s7	23	0x00000000		
\$t8	24	0x00000000		
\$t9	25	0x00000000		
\$k0	26	0x00000000		
\$k1	27	0x00000000		
\$gp	28	0x10008000		
\$sp	29	0x7ffffc		
\$fp	30	0x00000000		
\$ra	31	0x00000000		
pc		0x00400064		

可以看到，通过 CPU 得到的寄存器值和 MARS 中的一样，说明能运行正常

七、实验验证程序下板测试过程与实现

下图的七段数码管不断切换指令，说明 CPU 正在有序读取、处理指令，它的功能基本正常。





八、总结与体会

在本次实验中，我从最初的对 CPU 构架处于一知半解的状态，只是在理论层面有所了解到通过 vivado 做出 31 条指令 CPU，对 CPU 的设计流程、数据通路、控制器等有了更深刻的理解。

首先通过每条指令所需的部件画出数据通路，发现很多指令的数据通路是完全一样的；然后画出指令流程图进行 CPU 控制器的设计。前期准备工作完成后，开始代码环节。通过顶层模块调用 CPU，指令寄存器（设置 IP 核）和数据寄存器。在 CPU 中需要 ALU，regfile，PC 等模块实现 CPU 的功能。代码完成后，通过 IP 核读入 coe 文件，将寄存器结果写入文件与 MARS 编译后的结果对比，准确无误说明设计正确。在设计过程中遇到很多错误，主要是比对过程中寄存器结果、地址、程序计数器与 MARS 汇编的结果不一样。检查发现数据通路，微指令的逻辑表达式有误，重新分析、设计后解决了这些问题。

总之我从最初对 CPU 构架的模糊认识，到分析指令集，设计数据通路和控制器，最后实际操作逐步搭建一个能够运行指令的 CPU，更深入地理解了计算机系统的核心部件。

九、附件（所有程序）

CPU 的设计程序

9.1 sccomp_dataflow 模块，顶层模块，调用 cpu，IMEM，DMEM 三个模块

```
module sccomp_dataflow(
    input clk_in,
    input reset,
    output [31:0] inst,
    output [31:0] pc
);

/* CPU 用 */
wire [31:0] pc_out;
wire [31:0] dm_addr_temp;

/* IMEM 用 */
wire [31:0] im_addr_in;
wire [31:0] im_instr_out;

assign im_addr_in = pc_out - 32'h00400000;

/* DMEM 用 */
wire dm_ena;
wire dm_r, dm_w;
wire [31:0] dm_addr;
wire [31:0] dm_data_out;
wire [31:0] dm_data_w;
```

```

assign dm_addr = (dm_addr_temp - 32'h10010000)/4;

/* 输出用 */
assign pc = pc_out;
assign inst = im_instr_out;

/* IMEM 指令存储器调用 */
IMEM imem(.im_addr_in(im_addr_in[12:2]),.im_instr_out(im_instr_out));

/* DMEM 数据存储器调用 */
DMEM dmem(
    .dm_clk(clk_in),
    .dm_ena(dm_ena),
    .dm_r(dm_r),
    .dm_w(dm_w),
    .dm_addr(dm_addr[10:0]),
    .dm_data_in(dm_data_w),
    .dm_data_out(dm_data_out)
);

/* CPU 调用 */
cpu sccpu(
    .clk(clk_in),
    .ena(1'b1),
    .rst_n(reset),
    .instr_in(im_instr_out),
    .dm_data(dm_data_out),
    .dm_ena(dm_ena),
    .dm_w(dm_w),
    .dm_r(dm_r),
    .pc_out(pc_out),
    .dm_addr(dm_addr_temp),
    .dm_data_w(dm_data_w)
);

endmodule

```

9.2 cpu 模块//最重要的部件，负责调用 PC，Controler 等模块，并将各个模块连接起来

```

module cpu(

```

```

    input clk,                //CPU 执行时钟
    input ena,                //使能信号端
    input rst_n,              //复位信号
    input [31:0] instr_in,    //当前要执行的指令
    input [31:0] dm_data,     //读取到的 DMEM 的具体内容
    output dm_ena,            //是否需要启用 DMEM
    output dm_w,              //如果启用 DMEM, 是否为写入
    output dm_r,              //如果启用 DMEM, 是否为读取
    output [31:0] pc_out,     //输出指令地址, 告诉 IMEM 要取哪条
    output [31:0] dm_addr,    //启用 DMEM 的地址
    output [31:0] dm_data_w   //要写入 DMEM 的内容
);
/* Decoder 用 , 定义了 31 条指令发生信号*/
wire add_flag,  addu_flag, sub_flag, subu_flag, and_flag, or_flag, xor_flag,
nor_flag,
    slt_flag,  sltu_flag,
    sll_flag,  srl_flag,  sra_flag,  sllv_flag,
    srlv_flag, srav_flag,
    jr_flag,
    addi_flag, addiu_flag,
    andi_flag, ori_flag,  xori_flag,
    lw_flag,   sw_flag,
    beq_flag,  bne_flag,
    slti_flag, sltiu_flag,
    lui_flag,
    j_flag,    jal_flag;    //各个指令的标志信息
wire [4:0] RsC;            //Rs 对应的寄存器的地址
wire [4:0] RtC;            //Rt 对应的寄存器的地址
wire [4:0] RdC;            //Rd 对应的寄存器的地址
wire [4:0] shamt;          //位移偏移量 (SLL, SRL, SRA 用)
wire [15:0] immediate;     //立即数 (I 型指令用)
wire [25:0] address;       //跳转地址 (J 型指令用)

/* Control 用 */
wire reg_w;                //RegFile 寄存器堆是否可写入
wire [9:0] mux;            //9 个多路选择器的状态
wire [4:0] ext_ena;        //EXT 扩展是否开启, 5 个状态分别对应 EXT1、EXT5、
EXT16、EXT16(S)、EXT18(S), 其中 EXT[0] 对应 EXT1
wire cat_ena;              //是否需要拼接

/* ALU 用 */
wire [31:0] a, b;          //ALU 的 A、B 运算输入端
wire [3:0] aluc;           //ALUC 四位运算指令
wire [31:0] alu_data_out;  //ALU 输出的数据

```

```

wire zero, carry, negative, overflow;    //四个标志位

/* 寄存器堆 RegFile 用 */
wire [31:0] Rd_data_in;    //要向寄存器中写入的值
wire [31:0] Rs_data_out;    //Rs 对应的寄存器的输出值
wire [31:0] Rt_data_out;    //Rt 对应的寄存器的输出值

/* PC 寄存器用 */
wire [31:0] pc_addr_in;    //本次输入 PC 寄存器的指令地址，也就是下一次要执行的指令
wire [31:0] pc_addr_out;    //本次从 PC 寄存器中传出的指令地址，也就是当前需要执行的指令

/* 符号、数据扩展器线路 */
wire [31:0] ext1_out;
wire [31:0] ext5_out;
wire [31:0] ext16_out;
wire signed [31:0] ext16_out_signed;
wire signed [31:0] ext18_out_signed;

assign ext1_out = (slt_flag || sltu_flag) ? negative : (slti_flag || sltiu_flag) ? carry : 32'hz;
assign ext5_out = (sll_flag || srl_flag || sra_flag) ? shamt : 32'hz;
assign ext16_out = (andi_flag || ori_flag || xori_flag || lui_flag) ? { 16'h0 , immediate[15:0] } : 32'hz;
assign ext16_out_signed = (addi_flag || addiu_flag || lw_flag || sw_flag || slti_flag || sltiu_flag) ? { {16{immediate[15]}} , immediate[15:0] } : 32'hz;
assign ext18_out_signed = (beq_flag || bne_flag) ? {{14{immediate[15]}} , immediate[15:0] , 2'b0} : 32'hz;

/* ||拼接器线路 */
wire [31:0] cat_out;

assign cat_out = cat_ena ? {pc_out[31:28], address[25:0], 2'h0} : 32'hz;

/* NPC 线路 */
wire [31:0] npc;
assign npc = pc_addr_out + 4;

/* 多路选择器线路 */
wire [31:0] mux1_out;
wire [31:0] mux2_out;
wire [31:0] mux3_out;

```



```

wire [31:0] mux4_out;
wire [31:0] mux5_out;
wire [31:0] mux6_out;
wire [31:0] mux7_out;
wire [31:0] mux8_out;
wire [31:0] mux9_out;

assign mux1_out = mux[1] ? cat_out          : mux4_out;
assign mux2_out = mux[2] ? mux9_out         : dm_data;
assign mux3_out = mux[3] ? ext5_out         : ((sllv_flag || srlv_flag ||
srav_flag) ? { 27'h0, Rs_data_out[4:0] } : Rs_data_out); //特别注意如果是寄
存器的移位指令，要对进入 a 的数据进行处理，只取最后五位
assign mux4_out = mux[4] ? mux6_out        : Rs_data_out;
assign mux5_out = mux[5] ? mux8_out        : Rt_data_out;
assign mux6_out = mux[6] ? npc             : ext18_out_signed + npc;
assign mux7_out = mux[7] ? pc_addr_out + 4 : mux2_out;
assign mux8_out = mux[8] ? ext16_out_signed : ext16_out;
assign mux9_out = mux[9] ? alu_data_out    : ext1_out;

/* PC 线路 */
assign pc_addr_in = mux1_out;

/* ALU 接线口 */
assign a = mux3_out;
assign b = mux5_out;

/* IMEM 接口 */
assign pc_out = pc_addr_out;

/* DMEM 接口 */
assign dm_ena = (dm_r || dm_w) ? 1'b1 : 1'b0;
assign dm_addr = alu_data_out;
assign dm_data_w = Rt_data_out;

/* 寄存器堆线路 */
assign Rd_data_in = mux7_out;

/* 实例化译码器 */
Decoder Decoder_inst(
    .instr_in(instr_in),
    .add_flag(add_flag),
    .addu_flag(addu_flag),
    .sub_flag(sub_flag),
    .subu_flag(subu_flag),

```

```

.and_flag(and_flag),
.or_flag(or_flag),
.xor_flag(xor_flag),
.nor_flag(nor_flag),
.slt_flag(slt_flag),
.sltu_flag(sltu_flag),
.sll_flag(sll_flag) ,
.srl_flag(srl_flag),
.sra_flag(sra_flag),
.sllv_flag(sllv_flag),
.srlv_flag(srlv_flag),
.srav_flag(srav_flag),
.jr_flag(jr_flag),
.addi_flag(addi_flag),
.addiu_flag(addiu_flag),
.andi_flag(andi_flag),
.ori_flag(ori_flag),
.xori_flag(xori_flag),
.lw_flag(lw_flag),
.sw_flag(sw_flag),
.beq_flag(beq_flag),
.bne_flag(bne_flag),
.slti_flag(slti_flag),
.sltiu_flag(sltiu_flag),
.lui_flag(lui_flag),
.j_flag(j_flag),
.jal_flag(jal_flag),
.RsC(RsC),
.RtC(RtC),
.RdC(RdC),
.shamt(shamt),
.immediate(immediate),
.address(address)
);

```

/* 实例化控制器 */

```

Controler Controller_inst(
    .add_flag(add_flag),
    .addu_flag(addu_flag),
    .sub_flag(sub_flag),
    .subu_flag(subu_flag),
    .and_flag(and_flag),
    .or_flag(or_flag),
    .xor_flag(xor_flag),

```

```

.nor_flag(nor_flag),
.slt_flag(slt_flag),
.sltu_flag(sltu_flag),
.sll_flag(sll_flag) ,
.srl_flag(srl_flag),
.sra_flag(sra_flag),
.sllv_flag(sllv_flag),
.srlv_flag(srlv_flag),
.srav_flag(srav_flag),
.jr_flag(jr_flag),
.addi_flag(addi_flag),
.addiu_flag(addiu_flag),
.andi_flag(andi_flag),
.ori_flag(ori_flag),
.xori_flag(xori_flag),
.lw_flag(lw_flag),
.sw_flag(sw_flag),
.beq_flag(beq_flag),
.bne_flag(bne_flag),
.slti_flag(slti_flag),
.sltiu_flag(sltiu_flag),
.lui_flag(lui_flag),
.j_flag(j_flag),
.jal_flag(jal_flag),
.zero(zero),
.reg_w(reg_w),
.aluc(aluc),
.dm_r(dm_r),
.dm_w(dm_w),
.ext_ena(ext_ena),
.cat_ena(cat_ena),
.mux(mux)
);

```

/* 实例化 ALU */

```

ALU ALU_inst(
    .A(a),
    .B(b),
    .ALUC(aluc),
    .alu_data_out(alu_data_out),
    .zero(zero),
    .carry(carry),
    .negative(negative),
    .overflow(overflow)

```

```

    );

/* 实例化寄存器堆 */
regfile cpu_ref(
    .reg_clk(clk),
    .reg_ena(ena),
    .rst_n(rst_n),
    .reg_w(reg_w),
    .RdC(RdC),
    .RtC(RtC),
    .RsC(RsC),
    .Rd_data_in(Rd_data_in),
    .Rs_data_out(Rs_data_out),
    .Rt_data_out(Rt_data_out)
);

/* 实例化 PC 寄存器 */
PC PC_inst(
    .pc_clk(clk),
    .pc_ena(ena),
    .rst_n(rst_n),
    .pc_addr_in(pc_addr_in),
    .pc_addr_out(pc_addr_out)
);

endmodule

```

9.3 ALU 模块//算术逻辑单元模块，实现多种算术、逻辑运算

```

module ALU(
    input  [31:0] A,           //对应 A 接口
    input  [31:0] B,           //对应 B 接口
    input  [3:0] ALUC,         //ALUC 四位操作指令
    output [31:0] alu_data_out, //输出数据
    output zero,               //ZF 标志位，BEQ/BNE 使用
    output carry,              //CF 标志位，SLTI/SLTIU 使用
    output negative,           //NF(SF)标志位，SLT/SLTU 使用
    output overflow             //OF 标志位，其实没有用到
);
/* 定义各指令对应的操作 */
parameter ADDU = 4'b0000;
parameter ADD  = 4'b0010;
parameter SUBU = 4'b0001;
parameter SUB  = 4'b0011;

```

```

parameter AND = 4'b0100;
parameter OR = 4'b0101;
parameter XOR = 4'b0110;
parameter NOR = 4'b0111;
parameter LUI1 = 4'b1000;
parameter LUI2 = 4'b1001; //注意: LUI 是 100X, 因此有 LUI1 和 LUI2 之分
parameter SLT = 4'b1011;
parameter SLTU = 4'b1010;
parameter SRA = 4'b1100;
parameter SLL = 4'b1110; //SLL 和 SLA 本质上是一样的, 但是由于 SLL 和 SLR 的
//指令为 111X, 因此将其分开了
parameter SLA = 4'b1111;
parameter SRL = 4'b1101;
/* 定义一些内部用的变量 */
reg [32:0] result; //存储结果, 设置成 33 位是为了标志位的判断
wire signed [31:0] signedA, signedB; //由于 A 和 B 传进来是无符号的, 因此我们需要
//定义两个有符号 wire 型变量来存储 A 和 B 在有符号解释下的值
assign signedA = A;
assign signedB = B;

always @(*)
begin
    case(ALUC)
        ADDU:      begin result <= A + B; end
        ADD:      begin result <= signedA + signedB; end
        SUBU:      begin result <= A - B; end
        SUB:      begin result <= signedA - signedB; end
        AND:      begin result <= A & B; end
        OR:       begin result <= A | B; end
        XOR:      begin result <= A ^ B; end
        NOR:      begin result <= ~(A | B); end
        LUI1,LUI2: begin result <= { B[15:0] , 16'b0 }; end
        SLT:      begin result <= signedA - signedB; end
        SLTU:     begin result <= A - B; end
        SRA:      begin result <= signedB >>> signedA; end
        SLL,SLA:  begin result <= B << A; end
        SRL:      begin result <= B >> A; end
    endcase
end

assign alu_data_out = result[31:0];
assign zero = (result == 32'b0) ? 1 : 0;
assign carry = result[32];
assign negative = (ALUC == SLT?(signedA < signedB) : ((ALUC == SLTU) ? (A

```



```

< B) : 1'b0));
assign overflow = result[32];

endmodule

```

9.4 regfile 模块//寄存器堆，含有 32 个寄存器读写内容

```

module regfile(                                //寄存器堆 RegFile，写入为同步，读取为异步
    input  reg_clk,                            //时钟信号，下降沿有效
    input  reg_ena,                            //使能信号端，上升沿有效
    input  rst_n,                             //复位信号，高电平有效（检测上升沿）
    input  reg_w,                             //写信号，高电平时寄存器可写入，低电平不可写入
    input  [4:0] RdC,                          //Rd 对应的寄存器的地址（写入端）
    input  [4:0] RtC,                          //Rt 对应的寄存器的地址（输出端）
    input  [4:0] RsC,                          //Rs 对应的寄存器的地址（输出端）
    input  [31:0] Rd_data_in,                  //要向寄存器中写入的值（需拉高 reg_w）
    output [31:0] Rs_data_out,                 //Rs 对应的寄存器的输出值
    output [31:0] Rt_data_out                  //Rt 对应的寄存器的输出值
);
/* 内部用变量 */
reg [31:0] array_reg [31:0];                //定义寄存器堆

/* 赋值，异步读取 */
assign Rs_data_out = reg_ena ? array_reg[RsC] : 32'bz;
assign Rt_data_out = reg_ena ? array_reg[RtC] : 32'bz;

always @(negedge reg_clk or posedge rst_n) //复位信号上升沿或时钟下降沿有效
begin
    if(rst_n && reg_ena)
    begin
        array_reg[0]  <= 32'h0;
        array_reg[1]  <= 32'h0;
        array_reg[2]  <= 32'h0;
        array_reg[3]  <= 32'h0;
        array_reg[4]  <= 32'h0;
        array_reg[5]  <= 32'h0;
        array_reg[6]  <= 32'h0;
        array_reg[7]  <= 32'h0;
        array_reg[8]  <= 32'h0;
        array_reg[9]  <= 32'h0;
        array_reg[10] <= 32'h0;
        array_reg[11] <= 32'h0;
        array_reg[12] <= 32'h0;

```

```

        array_reg[13] <= 32'h0;
        array_reg[14] <= 32'h0;
        array_reg[15] <= 32'h0;
        array_reg[16] <= 32'h0;
        array_reg[17] <= 32'h0;
        array_reg[18] <= 32'h0;
        array_reg[19] <= 32'h0;
        array_reg[20] <= 32'h0;
        array_reg[21] <= 32'h0;
        array_reg[22] <= 32'h0;
        array_reg[23] <= 32'h0;
        array_reg[24] <= 32'h0;
        array_reg[25] <= 32'h0;
        array_reg[26] <= 32'h0;
        array_reg[27] <= 32'h0;
        array_reg[28] <= 32'h0;
        array_reg[29] <= 32'h0;
        array_reg[30] <= 32'h0;
        array_reg[31] <= 32'h0;
    end
    else if(reg_ena && reg_w && (RdC != 5'h0))
        array_reg[RdC] <= Rd_data_in;
    end
endmodule

```

9.5 PC 模块//采用异步读取同步写入方式声明一个 pc_reg[31:0]，为了与 MARS 中指令存储初始地址一致，让 PC 的初值是 32'h00400000

```

module PC(
    //指令地址寄存器
    input  pc_clk,           //PC 寄存器的时钟信号，写入为同步（读取为异步
    input  pc_ena,           //使能端信号，高电平有效
    input  rst_n,            //复位信号，高电平有效
    input  [31:0] pc_addr_in, //本次输入 PC 寄存器的指令地址，也就是下一次要执行的指令
    output [31:0] pc_addr_out //本次从 PC 寄存器中传出的指令地址，也就是当前需要执行的指令
);
    reg [31:0] pc_reg = 32'h00400000; //起始位置在 32'h00400000,
    assign pc_addr_out = pc_ena ? pc_reg : 32'hz;
    always @(negedge pc_clk or posedge rst_n) /
    begin
        if(rst_n && pc_ena)
            pc_reg <= 32'h00400000; //注意起始位置时 32'h00400000
        else if(pc_ena)

```

```

        pc_reg <= pc_addr_in;

end
endmodule

```

9.6 Decoder 模块//主要功能是译码

```

module Decoder(
    /*X_flag 表示输入指令是否为 X*/
    input  [31:0] instr_in,
    output add_flag,
    output addu_flag,
    output sub_flag,
    output subu_flag,
    output and_flag,
    output or_flag,
    output xor_flag,
    output nor_flag,
    output slt_flag,
    output sltu_flag,
    output sll_flag,
    output srl_flag,
    output sra_flag,
    output sllv_flag,
    output srlv_flag,
    output srav_flag,
    output jr_flag,
    output addi_flag,
    output addiu_flag,
    output andi_flag,
    output ori_flag,
    output xori_flag,
    output lw_flag,
    output sw_flag,
    output beq_flag,
    output bne_flag,
    output slti_flag,
    output sltiu_flag,
    output lui_flag,
    output j_flag,
    output jal_flag,
    output [4:0] RsC,
    output [4:0] RtC,
    output [4:0] RdC,

```

```

        output [4:0] shamt,
        output [15:0] immediate,
        output [25:0] address
    );
    /* 下面这些指令经过了扩展，OP 段全为 0，需要额外的 6 位 FUNC 加以区分 */
    parameter ADD_OPE    = 6'b100000;
    parameter ADDU_OPE   = 6'b100001;
    parameter SUB_OPE    = 6'b100010;
    parameter SUBU_OPE   = 6'b100011;
    parameter AND_OPE    = 6'b100100;
    parameter OR_OPE     = 6'b100101;
    parameter XOR_OPE    = 6'b100110;
    parameter NOR_OPE    = 6'b100111;
    parameter SLT_OPE    = 6'b101010;
    parameter SLTU_OPE   = 6'b101011;

    parameter SLL_OPE    = 6'b000000;
    parameter SRL_OPE    = 6'b000010;
    parameter SRA_OPE    = 6'b000011;

    parameter SLLV_OPE   = 6'b000100;
    parameter SRLV_OPE   = 6'b000110;
    parameter SRAV_OPE   = 6'b000111;

    parameter JR_OPE     = 6'b001000;
    /* 下面这些指令通过 OP 段直接加以区分 */
    parameter ADDI_OPE   = 6'b001000;
    parameter ADDIU_OPE  = 6'b001001;
    parameter ANDI_OPE   = 6'b001100;
    parameter ORI_OPE    = 6'b001101;
    parameter XORI_OPE   = 6'b001110;
    parameter LW_OPE     = 6'b100011;
    parameter SW_OPE     = 6'b101011;
    parameter BEQ_OPE    = 6'b000100;
    parameter BNE_OPE    = 6'b000101;
    parameter SLTI_OPE   = 6'b001010;
    parameter SLTIU_OPE  = 6'b001011;

    parameter LUI_OPE    = 6'b001111;

    parameter J_OPE      = 6'b000010;
    parameter JAL_OPE    = 6'b000011;

    /* 下面是赋值 */

```

```

/* 对指令进行译码，判断是哪个指令 */
assign add_flag    = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
ADD_OPE )) ? 1'b1 : 1'b0;
assign addu_flag   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
ADDU_OPE)) ? 1'b1 : 1'b0;
assign sub_flag    = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
SUB_OPE )) ? 1'b1 : 1'b0;
assign subu_flag   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
SUBU_OPE)) ? 1'b1 : 1'b0;
assign and_flag    = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
AND_OPE )) ? 1'b1 : 1'b0;
assign or_flag     = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
OR_OPE  )) ? 1'b1 : 1'b0;
assign xor_flag    = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
XOR_OPE )) ? 1'b1 : 1'b0;
assign nor_flag    = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
NOR_OPE )) ? 1'b1 : 1'b0;
assign slt_flag    = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
SLT_OPE )) ? 1'b1 : 1'b0;
assign sltu_flag   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
SLTU_OPE)) ? 1'b1 : 1'b0;

assign sll_flag    = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
SLL_OPE )) ? 1'b1 : 1'b0;
assign srl_flag    = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
SRL_OPE )) ? 1'b1 : 1'b0;
assign sra_flag    = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
SRA_OPE )) ? 1'b1 : 1'b0;

assign sllv_flag   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
SLLV_OPE)) ? 1'b1 : 1'b0;
assign srlv_flag   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
SRLV_OPE)) ? 1'b1 : 1'b0;
assign srav_flag   = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
SRAV_OPE)) ? 1'b1 : 1'b0;
assign jr_flag     = ((instr_in[31:26] == 6'h0) && (instr_in[5:0] ==
JR_OPE  )) ? 1'b1 : 1'b0;

assign addi_flag   = (instr_in[31:26] == ADDI_OPE ) ? 1'b1 : 1'b0;
assign addiu_flag  = (instr_in[31:26] == ADDIU_OPE) ? 1'b1 : 1'b0;
assign andi_flag   = (instr_in[31:26] == ANDI_OPE ) ? 1'b1 : 1'b0;
assign ori_flag    = (instr_in[31:26] == ORI_OPE  ) ? 1'b1 : 1'b0;
assign xori_flag   = (instr_in[31:26] == XORI_OPE ) ? 1'b1 : 1'b0;
assign lw_flag     = (instr_in[31:26] == LW_OPE   ) ? 1'b1 : 1'b0;

```

```

assign sw_flag    = (instr_in[31:26] == SW_OPE    ) ? 1'b1 : 1'b0;
assign beq_flag   = (instr_in[31:26] == BEQ_OPE   ) ? 1'b1 : 1'b0;
assign bne_flag   = (instr_in[31:26] == BNE_OPE   ) ? 1'b1 : 1'b0;
assign slti_flag  = (instr_in[31:26] == SLTI_OPE  ) ? 1'b1 : 1'b0;
assign sltiu_flag = (instr_in[31:26] == SLTIU_OPE ) ? 1'b1 : 1'b0;

assign lui_flag   = (instr_in[31:26] == LUI_OPE   ) ? 1'b1 : 1'b0;

assign j_flag     = (instr_in[31:26] == J_OPE     ) ? 1'b1 : 1'b0;
assign jal_flag   = (instr_in[31:26] == JAL_OPE   ) ? 1'b1 : 1'b0;

/* 取出指令中各部分的值 */
assign RsC = (add_flag  || addu_flag || sub_flag  || subu_flag ||
              and_flag  || or_flag   || xor_flag  || nor_flag  ||
              slt_flag  || sltu_flag || sllv_flag || srlv_flag ||
              srav_flag || jr_flag   || addi_flag || addiu_flag ||
              andi_flag || ori_flag  || xori_flag || lw_flag   ||
              sw_flag   || beq_flag  || bne_flag  || slti_flag  ||
              sltiu_flag) ? instr_in[25:21] : 5'hz;

assign RtC = (add_flag  || addu_flag || sub_flag  || subu_flag ||
              and_flag  || or_flag   || xor_flag  || nor_flag  ||
              slt_flag  || sltu_flag || sll_flag  || srl_flag  ||
              sra_flag  || sllv_flag || srlv_flag || srav_flag ||
              sw_flag   || beq_flag  || bne_flag ) ? instr_in[20:16] : 5'hz;

assign RdC = (add_flag  || addu_flag || sub_flag  || subu_flag ||
              and_flag  || or_flag   || xor_flag  || nor_flag  ||
              slt_flag  || sltu_flag || sll_flag  || srl_flag  ||
              sra_flag  || sllv_flag || srlv_flag || srav_flag) ?
instr_in[15:11] : ((
              addi_flag || addiu_flag || andi_flag || ori_flag  ||
              xori_flag || lw_flag   || slti_flag || sltiu_flag ||
              lui_flag) ? instr_in[20:16] : (jal_flag ? 5'd31 : 5'hz));

assign shamt = (sll_flag || srl_flag || sra_flag) ? instr_in[10:6] : 5'hz;

assign immediate = (addi_flag || addiu_flag || andi_flag || ori_flag ||
                    xori_flag || lw_flag   || sw_flag   || beq_flag ||
                    bne_flag  || slti_flag  || sltiu_flag || lui_flag) ?
instr_in[15:0] : 16'hz;

assign address = (j_flag || jal_flag) ? instr_in[25:0] : 26'hz;

```

```
endmodule
```

9.7 Controller 模块//根据当前指令生成各种信号，决定多路选择器的选择状态

```
module Controller(  
    /*与 Decoder 一样，X_flag 表示判断输入指令是否为 X*/  
    input add_flag,  
    input addu_flag,  
    input sub_flag,  
    input subu_flag,  
    input and_flag,  
    input or_flag,  
    input xor_flag,  
    input nor_flag,  
    input slt_flag,  
    input sltu_flag,  
    input sll_flag,  
    input srl_flag,  
    input sra_flag,  
    input sllv_flag,  
    input srlv_flag,  
    input srav_flag,  
    input jr_flag,  
    input addi_flag,  
    input addiu_flag,  
    input andi_flag,  
    input ori_flag,  
    input xori_flag,  
    input lw_flag,  
    input sw_flag,  
    input beq_flag,  
    input bne_flag,  
    input slti_flag,  
    input sltiu_flag,  
    input lui_flag,  
    input j_flag,  
    input jal_flag,  
    input zero,  
    /* 所有用到的元件和指令这里都会涉及到 */  
    output reg_w,           //RegFile 寄存器堆是否可写入  
    output [3:0] aluc,      //ALUC 的指令，决定 ALUC 执行何种操作  
    output dm_r,           //DMEM 是否可写入  
    output dm_w,           //是否从 DMEM 中读取数据  
    output [4:0] ext_ena,   //EXT 扩展是否开启，5 个状态分别对应 EXT1、EXT5、
```


EXT16、EXT16(S)、EXT18(S),其中 EXT[0]对应 EXT1

```
output cat_ena,          //是否需要拼接
output [9:0] mux         //9个多路选择器的状态（选择0还是选择1）
);
/* 下面是赋值，也就是根据要执行的操作决定各元器件的状态 */
assign reg_w = (!jr_flag && !sw_flag && !beq_flag && !bne_flag && !j_flag) ?
1'b1 : 1'b0;

assign aluc[3] = (slt_flag || sltu_flag || sllv_flag || srlv_flag ||
srav_flag || sll_flag || srl_flag || sra_flag ||
slti_flag || sltiu_flag || lui_flag) ? 1'b1 : 1'b0;
assign aluc[2] = (and_flag || or_flag || xor_flag || nor_flag ||
sllv_flag || srlv_flag || srav_flag || sll_flag ||
srl_flag || sra_flag || andi_flag || ori_flag ||
xori_flag) ? 1'b1 : 1'b0;
assign aluc[1] = (add_flag || sub_flag || xor_flag || nor_flag ||
slt_flag || sltu_flag || sllv_flag || sll_flag ||
addi_flag || xori_flag || slti_flag || sltiu_flag) ?
1'b1 : 1'b0;
assign aluc[0] = (sub_flag || subu_flag || or_flag || nor_flag ||
slt_flag || sllv_flag || srlv_flag || sll_flag ||
srl_flag || ori_flag || slti_flag || lui_flag ||
beq_flag || bne_flag) ? 1'b1 : 1'b0;
//aluc[0]中 SLLV、SLL、LUI 加不加均可

assign dm_r = lw_flag ? 1'b1 : 1'b0;
assign dm_w = sw_flag ? 1'b1 : 1'b0;

assign ext_ena[4] = (beq_flag || bne_flag) ? 1'b1 : 1'b0;
assign ext_ena[3] = (addi_flag || addiu_flag || lw_flag || sw_flag
||slti_flag||sltiu_flag)?1'b1:1'b0;
//EXT16(S)
assign ext_ena[2] = (andi_flag || ori_flag || xori_flag || lui_flag) ?
1'b1 : 1'b0; //EXT16
assign ext_ena[1] = (sll_flag || srl_flag || sra_flag) ? 1'b1 : 1'b0;
//EXT5
assign ext_ena[0] = (slt_flag || sltu_flag || slti_flag || sltiu_flag) ?
1'b1 : 1'b0; //EXT1

assign cat_ena = (j_flag || jal_flag) ? 1'b1 : 1'b0;

assign mux[9] = (add_flag || addu_flag || sub_flag || subu_flag ||
and_flag || or_flag || xor_flag || nor_flag ||
sll_flag || srl_flag || sra_flag || sllv_flag ||
```

```

        srlv_flag || srav_flag || lui_flag || addi_flag ||
        addiu_flag || andi_flag || ori_flag || xori_flag) ? 1'b1 :
1'b0;
assign mux[8] = (addi_flag || addiu_flag || lw_flag || sw_flag ||
        slti_flag || sltiu_flag) ? 1'b1 : 1'b0;
assign mux[7] = jal_flag ? 1'b1 : 1'b0;
assign mux[6] = beq_flag ? ~zero : (bne_flag ? zero : 1'b1);
assign mux[5] = (addi_flag || addiu_flag || andi_flag || ori_flag ||
        xori_flag || lw_flag || sw_flag || slti_flag ||
        sltiu_flag || lui_flag) ? 1'b1 : 1'b0;
assign mux[4] = (!jr_flag && !j_flag && !jal_flag) ? 1'b1 : 1'b0;
assign mux[3] = (sll_flag || srl_flag || sra_flag) ? 1'b1 : 1'b0;
assign mux[2] = !lw_flag ? 1'b1 : 1'b0;
assign mux[1] = (j_flag || jal_flag) ? 1'b1 : 1'b0;

endmodule

```

9.8 DMEM 模块//实现数据存储器，采用异步读取同步写入

```

module DMEM(
    input dm_clk,                //DMEM 时钟信号，只在写数据时使用
    input dm_ena,                //使能信号端，高电平才读取/写入
    input dm_r,                  //read 读信号，读取时拉高
    input dm_w,                  //write 写信号，写入时拉高
    input [10:0] dm_addr,        //11 位地址，要读取/写入的地址
    input [31:0] dm_data_in,     //写入时要写入的数据
    output [31:0] dm_data_out    //读取时读取到的数据
);

reg [31:0] dmem [31:0]; //DMEM 区域

assign dm_data_out = (dm_ena && dm_r && !dm_w) ? dmem[dm_addr] : 32'bz;
/*不满足 dm_ena && dm_r && !dm_w 为 32'bz 高阻抗*/
always @(negedge dm_clk) //时钟上升沿写入数据
begin
    if(dm_ena && dm_w && !dm_r)
        dmem[dm_addr] <= dm_data_in;
end
endmodule

```

9.9 IMEM 模块//指令存储器，需要用到 Distributed Memory Generator IP 核

```

module IMEM(
    input [10:0] im_addr_in,     //11 位指令码地址，从 IMEM 中读指令

```

```

        output [31:0] im_instr_out    //32 位指令码
    );

    //实例化 IP 核，输入指令码地址返回对应的指令
    dist_mem_gen_0 imem(.a(im_addr_in), .spo(im_instr_out));
endmodule

```

9.10 testbench 模块//将 regfile 中 32 个寄存器状态和指令地址写入到文件中，以供后续分析

```

module cpu_tb;
reg clk;           //时钟信号
reg rst;           //复位信号
wire [31:0] inst;  //要执行的指令
wire [31:0] pc;    //下一条指令的地址
reg [31:0] cnt;    //计数器，已经执行了几条指令
integer file_open;

initial
begin
    clk = 1'b0;
    rst = 1'b1;
    #10 rst = 1'b0;
    cnt = 0;
end

always #10 clk = ~clk;

always @ (posedge clk) begin
    cnt <= cnt + 1'b1;
    file_open = $fopen("D:\\output.txt", "a+");
    $fdisplay(file_open, "OP: %d", cnt);
    $fdisplay(file_open, "Instr_addr = %h", sc_inst.inst);
    $fdisplay(file_open, "$zero= %h", sc_inst.sccpu.cpu_ref.array_reg[0]);
    $fdisplay(file_open, "$at  = %h", sc_inst.sccpu.cpu_ref.array_reg[1]);
    $fdisplay(file_open, "$v0  = %h", sc_inst.sccpu.cpu_ref.array_reg[2]);
    $fdisplay(file_open, "$v1  = %h", sc_inst.sccpu.cpu_ref.array_reg[3]);
    $fdisplay(file_open, "$a0  = %h", sc_inst.sccpu.cpu_ref.array_reg[4]);
    $fdisplay(file_open, "$a1  = %h", sc_inst.sccpu.cpu_ref.array_reg[5]);
    $fdisplay(file_open, "$a2  = %h", sc_inst.sccpu.cpu_ref.array_reg[6]);
    $fdisplay(file_open, "$a3  = %h", sc_inst.sccpu.cpu_ref.array_reg[7]);
    $fdisplay(file_open, "$t0  = %h", sc_inst.sccpu.cpu_ref.array_reg[8]);
    $fdisplay(file_open, "$t1  = %h", sc_inst.sccpu.cpu_ref.array_reg[9]);

```

```

    $fdisplay(file_open, "$t2 = %h", sc_inst.sccpu.cpu_ref.array_reg[10]);
    $fdisplay(file_open, "$t3 = %h", sc_inst.sccpu.cpu_ref.array_reg[11]);
    $fdisplay(file_open, "$t4 = %h", sc_inst.sccpu.cpu_ref.array_reg[12]);
    $fdisplay(file_open, "$t5 = %h", sc_inst.sccpu.cpu_ref.array_reg[13]);
    $fdisplay(file_open, "$t6 = %h", sc_inst.sccpu.cpu_ref.array_reg[14]);
    $fdisplay(file_open, "$t7 = %h", sc_inst.sccpu.cpu_ref.array_reg[15]);
    $fdisplay(file_open, "$s0 = %h", sc_inst.sccpu.cpu_ref.array_reg[16]);
    $fdisplay(file_open, "$s1 = %h", sc_inst.sccpu.cpu_ref.array_reg[17]);
    $fdisplay(file_open, "$s2 = %h", sc_inst.sccpu.cpu_ref.array_reg[18]);
    $fdisplay(file_open, "$s3 = %h", sc_inst.sccpu.cpu_ref.array_reg[19]);
    $fdisplay(file_open, "$s4 = %h", sc_inst.sccpu.cpu_ref.array_reg[20]);
    $fdisplay(file_open, "$s5 = %h", sc_inst.sccpu.cpu_ref.array_reg[21]);
    $fdisplay(file_open, "$s6 = %h", sc_inst.sccpu.cpu_ref.array_reg[22]);
    $fdisplay(file_open, "$s7 = %h", sc_inst.sccpu.cpu_ref.array_reg[23]);
    $fdisplay(file_open, "$t8 = %h", sc_inst.sccpu.cpu_ref.array_reg[24]);
    $fdisplay(file_open, "$t9 = %h", sc_inst.sccpu.cpu_ref.array_reg[25]);
    $fdisplay(file_open, "$k0 = %h", sc_inst.sccpu.cpu_ref.array_reg[26]);
    $fdisplay(file_open, "$k1 = %h", sc_inst.sccpu.cpu_ref.array_reg[27]);
    $fdisplay(file_open, "$gp = %h", sc_inst.sccpu.cpu_ref.array_reg[28]);
    $fdisplay(file_open, "$sp = %h", sc_inst.sccpu.cpu_ref.array_reg[29]);
    $fdisplay(file_open, "$fp = %h", sc_inst.sccpu.cpu_ref.array_reg[30]);
    $fdisplay(file_open, "$ra = %h", sc_inst.sccpu.cpu_ref.array_reg[31]);
    $fdisplay(file_open, "$pc = %h\n", sc_inst.pc);
    $fclose(file_open);
end

sccomp_dataflow sc_inst(
    .clk_in(clk),
    .reset(rst),
    .inst(inst),
    .pc(pc)
);

endmodule

```